
Introduction to Reinforcement Learning (EE675):

Assignment 1

Q. 1: Consider a predator-prey game in a $N \times N$ grid. Each cell in this grid is identified by a pair of numbers $(x, y) \in \{1, \dots, N\} \times \{1, \dots, N\}$. The time t is discrete and takes values in $\{0, 1, \dots\}$. At the initial time $t = 0$, the predator is at $(1, 1)$ and the prey is at (N, N) . At each instant, the prey can either remain still or move to one of its neighboring cells with uniform probability. Your task is to model the predator's optimal attacking strategy. The predator (the learner in this case) can either remain still or move to one of the neighboring cells in each instant. If the predator catches the prey (i.e., they are in the same cell), the predator receives a reward of $+1$; otherwise, it receives a zero reward. Once the predator catches the current prey, it dies and a new prey pops up uniformly randomly in one of remaining $N^2 - 1$ cells, and the game continues. Take the discount factor $\gamma = 0.99$.

(a) Write a simulator function in Python that takes the following inputs:

- the value of N
- the current location of the predator
- the current location of the prey
- the action taken by the predator

and generates the following outputs:

- location of the predator in the next time step
- the location of the prey in the next time step
- the reward value indicating whether the predator was able to catch the prey

While coding, carefully take all edge cases into account. For example, note that the set of actions that the predator can take near the corner or the boundary of the grid is different from that it can take near the center. One way to handle this state-dependent action set is not to change the location of the predator for *invalid* actions. For example, if the predator is at the bottom left corner and its chosen action is moving down, then the simulator function should not change the location of the predator in the next time step.

- (b) Write a kernel function in Python that takes N as an input and generates the state transition kernel corresponding to the above environment as an output. Recall that state-transition kernel must be a matrix of size $|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}|$, where $|\mathcal{S}|$ and $|\mathcal{A}|$ denote the sizes of the state space and action space, respectively. Carefully choose the state and action spaces. One easy way to verify if the generated matrix is indeed a state-transition kernel is by checking whether the elements in each row add up to 1.
- (c) Write a reward function in Python that takes N as an input and generates the reward function of the above environment (represented as a matrix of size $|\mathcal{S}| \times |\mathcal{A}|$) as an output.
- (d) Consider the following (stationary) policy of the predator. The predator, with probability 0.5, moves to that cell that minimizes its distance to the current location of the prey (measured by the minimum number of cells it has to cross to reach the current location of the prey), and with combined probability 0.5, it takes other valid actions uniformly randomly. Write a `sample_policy` function in Python that takes N as an input and generates the above policy as a matrix of size $|\mathcal{S}| \times |\mathcal{A}|$.
- (e) Write a `induced_kernel` function in Python that takes the following two inputs:

- state-transition kernel (a matrix of size $|\mathcal{S}||\mathcal{A}| \times |\mathcal{S}|$)
- a policy function (a matrix of size $|\mathcal{S}| \times |\mathcal{A}|$)

and generates the following output:

- the policy-induced transition kernel (a matrix of size $|\mathcal{S}| \times |\mathcal{S}|$)

We discussed the idea of policy-induced kernel in the class in the context of stationary deterministic policies. You need to extend this concept to stationary non-deterministic policies.

- (f) Write a `induced_reward` function in Python that takes the following two inputs:

- reward-function (a matrix of size $|\mathcal{S}| \times |\mathcal{A}|$)
- a policy function (a matrix of size $|\mathcal{S}| \times |\mathcal{A}|$)

and generates the following output:

- the policy-induced reward function (a matrix of size $|\mathcal{S}| \times 1$)

We discussed the idea of policy-induced reward function in the class in the context of stationary deterministic policies. You need to extend this concept to stationary non-deterministic policies.

(g) Write a `state_value_eval` function in Python that takes the following input:

- a policy function
- a state-transition kernel
- a reward function

and generates a state value function (represented as a matrix of size $|\mathcal{S}| \times 1$) corresponding to the input policy as an output. Instead of using the matrix inversion-based method (which is computationally expensive for large N) taught in the class, come up with a recursive method to compute the value function.

(h) Write a `q_value_eval` function in Python that takes the following input:

- a policy function
- a state-transition kernel
- a reward function

and generates a Q function (represented as a matrix of size $|\mathcal{S}| \times |\mathcal{A}|$) corresponding to the input policy as an output.

(i) Prepare a PDF document that explains the principles behind each functions mentioned above.

(j) Using the above functions, evaluate the state values corresponding to the initial state $((1, 1), (N, N))$ and the sample policy mentioned in part (d) for $N = 5, 10, 15, 20, 25$. Plot these numbers in a graph (y-axis: state values, x-axis: N). Also, plot the corresponding run times (in seconds) in a separate graph (y-axis: run time, x-axis: N). Include these graphs in the PDF document.

General Instruction: Put each function in separate files. Name each variables, functions, and files appropriately. A readme.md file should clearly explain how to run the code to generate the two plots mentioned in part (j). Marks might be deducted due to lack of clarity.

[10]

Q. 2: This is a continuation of Q. 1.

(a) Write a `value_iteration` function that takes the following inputs:

- a state-transition kernel
- a reward function

and generates an approximately optimal Q function as an output via the value iteration algorithm. You may choose an appropriate approximation threshold ϵ .

(b) Write an `induced_policy` function that takes a Q function as an input and greedily extracts the policy function induced by it as an output.

(c) Write a `policy_iteration` function that takes the following inputs:

- a state-transition kernel
- a reward function

and generates an approximately optimal Q function as an output via the policy iteration algorithm. You may appropriately choose an appropriate approximation threshold ϵ .

(d) Run the `value_iteration` and `policy_iteration` functions for the problem described in Q. 1. Plot the L_1 -difference between the approximately optimal value functions obtained via these two procedures as a function of N for $N = 5, 10, 15, 20, 25$.

(e) Plot another graph describing the total run times (in seconds) as a function of N . Include these two plots in a PDF file. In the same file, please explain the working principles, naming conventions, etc. for each part of this question.

General Instruction: Each function should be written in separate files. A readme.md file should clearly explain how to run the code to generate the above plots. Marks might be deducted for lack of clarity.

[10]

Q. 3: This is a continuation of Q. 2. In this question, we will assume that we do not have access to the kernel function that provides the exact values of the state transition probabilities.

(a) Write an `estimate_kernel` function that calls the simulator function (defined in Q. 1(a)) $|\mathcal{S}||\mathcal{A}|K$ number of times (where K is an input to this function), and generates an estimated state-transition kernel as an output.

- (b) Run the `value_iteration` function (defined in Q. 2(a)) for the estimated kernels of the environment described in Q. 1 for $N = 5$ and $K = 5, 10, 15, 20, 25$. Calculate the L_1 difference between the approximately optimal Q function obtained for the estimated kernel and that obtained for the exact kernel as a function of K . Note that the approximately optimal Q function for the exact kernel is already computed in Q. 2(d).
- (c) Repeat (a) and (b) for 5 different randomization (or seed values), and plot the mean value and the standard deviation of the L_1 difference of the Q functions as a function of K . Include these two graphs in a PDF file. Please include the working principles, naming conventions, etc. for all parts of this question in the same PDF file.

General Instruction: A `readme.md` file should clearly explain how to run the code to generate the above plot. Marks might be deducted for lack of clarity.

[10]